DutchBay EPC Model — Complete Module Catalog

Every source module in `arunakulat/dutchbay-epc-model` (engine v15.3.0 @ a50b0bfce8e8), documented and classified for feasibility-report coverage. **294 modules** across 45 subsystems. Coverage legend: ● fired in the feasibility run · ● feasibility-relevant but not fired this pass · ○ not applicable to a wind feasibility.

Totals: ● 92 fired · ● 108 available-not-fired · ○ 94 not-applicable.

> Coverage rationale and the "why not included" analysis are in the companion **FEASIBILITY_COVERAGE.md**.

`finance` (23 — ●19 ●1 ○3)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	●	Package initializer for the v14 finance stack; documents that there is no standalone refinancing engine (balloon refinance lives inline in debt_v14) and that the retired refinancing modules had no consumers.	
`bess_lcos.py`	library	○	Read-only Levelised Cost of Storage (LCOS = PV(lifetime costs)/PV(lifetime discharged energy), discounted at project WACC) reporting view for a BESS; strictly off the cashflow path and KPI-neutral.	compute_lcos
`bess_project_economics.py`	library	○	Standalone BESS project economics: landed capex from a CIF quote + SL customs/FX layer, the CEB effective-cost curve, the separate charging-cost lever, and a standalone BESS cashflow -> IRR/NPV/LCOS.	standalone BESS project-economics entry point (landed-capex / CEB-cost / IRR-NPV-LCOS result)
`bess_revenue.py`	library	●	Resolves type: bess technologies and computes their availability-based capacity-charge revenue (and augmentation capex / SoH degradation) for the v14 cashflow; returns None/0.0 when no BESS block exists so wind-only runs are byte-identical.	resolve_bess_specs, bess_revenue_lkr_for_year, bess_augmentation_capex_lkr_for_year, mdsc_soh_for_year
`cashflow_v14.py`	library	●	Canonical v14 CFADS / annual-cashflow engine: builds annual CFADS and per-year rows (production, revenue, opex, tax, FX, statutory deductions) that debt sizing, DSCR and equity run on.	validate_parameters, build_annual_cfads, build_annual_rows, build_annual_rows_efficient, calculate_single_year_cfads
`cashflow_v14_contracts.py`	contract	●	Frozen dataclass contracts for the cashflow engine: CashflowParams (normalized numeric parameter surface) and FxHedge (resolved FX-forward hedge state).	CashflowParams, FxHedge
`cashflow_v14_fx.py`	library	●	Builds the FX curve (LKR per USD) from explicit/parametric config and applies FX-forward hedging when threading LKR CFADS into the USD-numeraire view.	_fx_curve, _hedged_usd, _resolve_fx_hedge
`cashflow_v14_params.py`	library	●	Normalizes raw scenario config into CashflowParams and validates parameters (capacity/CF paths, self-curtailment composition, indirect taxes) for the cashflow engine.	_build_cashflow_params, validate_parameters
`cashflow_v14_production.py`	library	●	Computes annual net energy production (degradation, grid loss, curtailment), tariff revenue, opex and statutory deductions; resolves per-tech generation specs and guards BESS capex declaration.	calculate_net_production_for_year, resolve_tech_generation_specs, validate_storage_capex_declared, _calculate_revenue_lkr, _calculate_opex_lkr, _calculate_statutory_deductions, _apply_risk_haircut
`cashflow_v14_tax.py`	library	●	Single canonical Sri Lanka tax engine for the cashflow: corporate income tax, straight-line depreciation (plant/civil split), vintage-tracked loss carry-forward, and interest/dividend withholding tax.	TaxConfig, TaxProfile, TaxResult, DepreciationSchedule, build_tax_profile, build_tax_series, calculate_tax
`cashflow_v14_utils.py`	library	●	Coercion and config-path helpers for the cashflow engine (fail-loud as_float/as_int, get_nested, percent-to-decimal, capacity/CF path candidate scans).	as_float, get_nested, as_int, as_int_or_none, _as_float_or_none, _pct_to_decimal, _resolve_first, CAPACITY_FACTOR_PATHS, CAPACITY_MW_PATHS
`debt_v14.py`	library	●	Debt planning engine: sizes debt, builds the amortization/DSCR-sculpted schedule, and resolves balloon treatment (cash_sweep/refinance/bullet/amortize) including the inline refinance stream.	debt schedule/sizing and balloon-resolution functions (_balloon_resolution_stream, _refinance_terms, schedule/sizing entry points)
`epc_helper_v14.py`	library	●	Lender-grade EPC/capex helper: breaks out EPC total (base, freight, contingency), converts to LKR via resolved FX, and registers EPC schema fields for the strict config guard.	epc_breakdown_from_config (+ EPC-total resolution and schema-registration helpers)

Module	Kind	Cov	Purpose	Key API
`epc_margin.py`	library	○	Standalone EPC construction-margin economics (e.g. the Kolonnawa BESS EPC tender): computes contract value, gross margin, return-on-cost, month-by-month cashflow, peak working capital and construction IRR.	EpcMarginResult (+ EPC-margin computation entry point)
`equity_distribution_v14_hydra.py`	library	●	Turns canonical v14 pipeline output (config, annual_rows, debt_result, kpis) into a JSON-safe equity distribution waterfall, delegating IRR/NPV via equity_v14.	EquityDistributionConfig, build_equity_distribution_schedule, calculate_equity_distribution_from_pipeline
`equity_v14.py`	library	●	Computes core equity investor metrics (equity IRR/NPV, cash-on-cash, MOIC, payback, PE triad) from an equity cashflow series, delegating IRR/NPV to finance.irr.	EquityCashflowSummary, summarise_equity_cashflows, calculate_equity_irr, calculate_equity_npv, calculate_cash_on_cash, calculate_moic, calculate_payback_period, calculate_pe_triad, calculate_equity_performance
`import_levies.py`	library	●	Single source of the capex/opex uplift arithmetic for Sri Lankan import levies and unrecoverable VAT via the opt-in taxes_indirect config block; absent the block it is byte-identical (all lines zero).	capex_uplift_from_config, compute_capex_uplift_usd, resolve_indirect_taxes
`irr.py`	library	●	Singleton IRR/NPV engine (ARCH-02): periodic NPV/IRR, date-aware XNPV/XIRR, configurable search bounds, robust solvers with bisection fallback and NaN/non-bracketing handling.	irr, npv, xirr, xnpv, approx_project_irr
`irr_config.py`	config	●	Helper utilities to load canonical YAML config and extract IRR validation bounds (Pattern 1 explicit-config integration) for the IRR engine.	load_config (+ IRR-bounds extraction helpers)
`self_curtailment_v14.py`	library	●	D6b seam that composes ONLY the self-curtailed fraction of a gated QSTS grid study into the finance curtailment loss key; the sole KPI-moving grid seam, default-off (returns 0.0 unless finance-wiring opt-in is set).	resolve_self_curtailment_decimal, compose_curtailment
`tech_types.py`	library	●	Single source of truth for technology TYPE discriminators (generation vs storage frozensets, validated-vs-enum-only gating, hybrid derivation) so a new generation tech aggregates everywhere without code change.	GENERATION_TYPES, STORAGE_TYPES, is_generation_type, is_modelled_generation_type, is_storage_type (+ allow_unvalidated_flat_cf gate)
`utils.py`	library	●	Consolidated lenient utility helpers for the finance module (get_nested, as_float/as_int with fallback defaults) used by debt/epc/levy code paths.	get_nested, as_float, as_int
`wacc_v14.py`	library	●	Project-specific WACC engine: CAPM cost of equity with beta de/re-levering, after-tax nominal and real WACC, prudential bump, and full component breakdown from a YAML config surface.	WACC computation entry points (CAPM/fixed/build-up modes + component breakdown)

`finance/cashflow` (1 — ●0 ●1 ○0)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	●	Placeholder public namespace package for the consolidated cashflow package; **all** is currently empty and it imports nothing (the source of truth remains the cashflow_v14* flat modules).	

`finance/equity` (1 — ●0 ●1 ○0)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	●	Public equity namespace package: re-exports the equity metrics from equity_v14 and the distribution-waterfall API from equity_distribution_v14_hydra.	EquityCashflowSummary, EquityDistributionConfig, build_equity_distribution_schedule, calculate_equity_distribution_from_pipeline, calculate_equity_irr, calculate_equity_npv, calculate_cash_on_cash, calculate_moic, calculate_payback_period, calculate_pe_triad, calculate_equity_performance, summarise_equity_cashflows

finance/grid (2 — ●0 ●2 ○0)

Module	Kind	Cov	Purpose	Key API
__init__.py	library	●	Package init for the finance-layer grid helpers: holds only grid type discriminators and the allow_unvalidated_grid opt-in gate; nothing here is imported by the finance cashflow engine (KPI-neutral).	
grid_types.py	library	●	Single source of truth for grid interconnection type discriminators: GFL/GFM converter classification, Type-3 DFIG vs Type-4 full-converter wind checks, assumption-basis provenance, and the allow_unvalidated_grid gate.	is_gfl_converter, is_gfm_capable_converter, is_type3_dfig, is_type4_full_converter, is_modelled_grid_tech, is_estimated_assumption_basis, allow_unvalidated_grid

analytics (top-level) (35 — ●25 ●10 ○0)

Module	Kind	Cov	Purpose	Key API
__init__.py	library	●	Package init that re-exports the v14 analytics contracts, returns/risk-metrics helpers, and FX contracts so callers import them from one namespace.	re-exports: ScenarioResult, CasperResult, WaccResult, MonteCarloResult, calculate_irr, calculate_npv, TailRiskAnalyzer, FXCurveOutput, etc.
aep_provenance.py	library	●	Config-first LIVE guard that folds the dormant AEP power-curve provenance control (approved/certified sources, placeholder detection) into the financed path, running at authored-scenario load and the API boundary; a pure detector that raises or no-ops and changes no number.	enforce_aep_provenance, resolve_provenance_policy, register_scenario_approved_sources, default_provenance_policy, ProvenancePolicy, AepProvenanceError
aep_reconciliation.py	library	●	Config-first detector that fails loud when a scenario's capacity_mw x capacity_factor x 8760 diverges beyond tolerance from a declared bankable net AEP (expected_results / aep_summary_path), resolving capacity/CF through the same engine paths; changes no computed number.	reconcile_capacity_factor_with_bankable_aep, reconcile_frozen_p90_with_bankable_summary, resolve_tolerance_pct, collect_bankable_net_aep_gwh, collect_bankable_net_aep_p90_gwh, resolve_billed_capacity_and_factor, AepReconciliationError
capital_risk_layer_v14.py	library	●	Lender-grade capital-risk facade (#33) that aggregates Monte-Carlo trial distributions into VaR/CVaR on equity IRR and NPV, min-DSCR/breach probability, and an NPV-distribution PNG.	build_capital_risk_report_from_trials, build_capital_risk_report_from_mc_result, compute_capital_risk_layer, emit_npv_distribution_from_trials, CapitalRiskReport, CapitalRiskLayer
capital_structure_optimizer_v14.py	library	●	Two opt-in capital-structure optimizers (#740): search the debt-tranche mix and the capex-contingency fraction to maximize a return objective subject to covenant constraints, scoring each candidate via evaluate_with_overrides.	optimize_debt_mix, optimize_capex_contingency
conditions_precedent.py	library	●	Config-first conditions-precedent (CP) checklist for first-drawdown/financial-close: validates each named CP status (satisfied/waived/pending) and rolls up the count of outstanding CPs that gate drawdown.	build_cp_report, validate_conditions_precedent, resolve_cp_policy, load_cp_taxonomy, CpReport, CpItem, CpFinding
config_schema.py	schema	●	Registry of required-field specifications per module and a helper to build a schema DataFrame; underpins the strict config-validation contract (CESSPIT).	RequiredFieldSpec, register_required_fields, get_required_fields, build_schema_dataframe
contracts_v14.py	contract	●	Central frozen-dataclass contracts (CCCDIR) for all v14 analytics result types — scenario, WACC, debt, sensitivity, Monte Carlo, CASPER, grid, capital-structure — with a model_dump()-compatible serialization facade.	ScenarioResult, WaccResult, TrancheDebtProfile, DebtCovenantSnapshot, MonteCarloResult, CasperResult, TornadoResult, SensitivitySuite, GridStudyResult, CapitalStructureOptimizationResult, CASPER_CONTRACT_VERSION
development_readiness.py	library	●	Config-first development-readiness / E&S status register: validates each workstream's Red/Amber/Green status and rolls up to the worst declared status (conservative lender view).	build_readiness_report, validate_development_readiness, resolve_readiness_policy, load_readiness_taxonomy, ReadinessReport, ReadinessItem
dscr_sensitivity.py	library	●	Dual-DSCR debt-sizing sensitivity analyzer: one-way/tornado sweeps showing how debt capacity varies with degradation, AEP, tariff, and when the P99 vs P50 DSCR constraint binds; has a Hydra CLI main().	analyze_dscr_sensitivity, analyze_single_variable, SensitivityConfig, main
evaluate_scenario.py	library	●	Coordinator-only single-scenario entry point: loads a base config, applies nested overrides, calls run_v14_pipeline once, and flattens engine outputs into a KPI dict for analytics tools.	evaluate_with_overrides
evaluation_v14.py	library	●	Canonical single evaluation gateway for analytics layers; applies dotted/nested overrides, runs the pipeline once, normalizes KPIs, and provides the CASPER tail-risk entry point.	evaluate_with_overrides, evaluate_scenario_from_dict, evaluate_with_casper_tail_risk, expand_dotted_overrides, normalize_kpi_dict, run_monte_carlo_analysis
evidence_register.py	library	●		

Module	Kind	Cov	Purpose	Key API
			Config-first assumption evidence register (lender assumption-provenance control): validates a scenario's evidence_register block against the tier taxonomy and reports covered vs missing material assumptions.	build_evidence_report, validate_evidence_register, resolve_evidence_policy, load_taxonomy, EvidenceReport, EvidenceFinding
`evidence_score.py`	library	●	Scoring layer over the evidence register: computes a single 0-100 bankability evidence-completeness score plus a band label from register coverage and per-tier strength weights.	build_evidence_score, load_evidence_score_weights, EvidenceScore, AssumptionScore, ScoreBand
`executive_workbook.py`	emitter	●	Builds the lender-facing single-scenario Executive Workbook xlsx (Summary/Cashflow/DebtService/Ratios/ScenarioSummary + optional ResourceTrend) from a live pipeline result, and normalizes it to byte-reproducible.	build_executive_workbook, frames_from_pipeline_result, emit_executive_workbook_from_pipeline, serialize_resource_trend, resource_trend_df_from_wind_export, resource_trend_df_from_solar_export
`export_helpers.py`	emitter	●	Excel/chart export utilities for scenario analytics: ExcelExporter (styled multi-sheet board workbooks), ChartExporter and ChartGenerator (matplotlib DSCR/IRR/NPV-distribution plots), plus pre-export validation and the shared DSCR highlight threshold.	ExcelExporter, ChartExporter, ChartGenerator, validate_for_export, DSCR_HIGHLIGHT_THRESHOLD
`feasibility_sections.py`	library	●	Config-first schema for the 20-section IC/DFI feasibility-report taxonomy: validates a scenario's declared section coverage (complete/draft/not_applicable) and rolls up missing sections per group.	build_feasibility_report, validate_feasibility_sections, resolve_feasibility_policy, load_feasibility_taxonomy, FeasibilityReport, FeasibilitySection
`fx_sensitivity_real.py`	library	●	Real-engine FX sensitivity analyzer sweeping FX-rate shocks, hedge ratios, and spread-bps deltas through the live pipeline (evaluate_with_overrides) to fit linear sensitivity coefficients on a target KPI.	FXSensitivityAnalyzer, FXSensitivityConfig, RealFXSensitivityResult, SensitivityCoefficient, FXSensitivityResult, FXSensitivityPoint
`infeasibility_diagnostics.py`	library	●	Structured infeasibility diagnostics and optimization audit log (#741): on a failed optimizer solve, reports which covenant bound was binding, by how much, and how hard the search looked.	build_capital_structure_diagnostics, build_parameter_diagnostics, audit_log_from_diagnostics, ConstraintViolation, InfeasibilityDiagnostics, OptimizationAuditLog
`irr_bridge.py`	library	●	Builds the disclosure-only project->equity IRR 'bridge' (leverage, cost-of-debt, tax-shield legs + a residual) reconciling the engine's published unlevered and levered IRRs; delegates all IRR math to finance.irr and result types to contracts_v14.	build_project_equity_irr_bridge, build_project_equity_irr_bridge_from_run
`monte_carlo_v14.py`	legacy	●	Deprecated backward-compat shim re-exporting MonteCarloEngine and run_monte_carlo_analysis from the canonical analytics.mc.engine.	MonteCarloEngine, run_monte_carlo_analysis (re-exported)
`optimization_v14.py`	library	●	Thin single-parameter optimizer facade over evaluate_with_overrides: maximizes/minimizes a target KPI across [lower, upper] (grid or SciPy bounded scalar) subject to an optional KPI constraint (e.g. max equity IRR s.t. min DSCR >= covenant).	optimize_parameter, OptimizationResult, OptimizationPoint
`output_paths.py`	library	●	Config-first single resolver for the v14 entrypoints' default artifact/report/export output roots, with optional run-scoped subdirectory grouping (default-off, byte-identical).	resolve_output_dir, default_run_id
`pipeline_analytics_v14.py`	library	●	Optional analytics wrapper over the base pipeline that adds config-gated returns and tail-risk analysis (VaR/CVaR) around run_v14_pipeline.	run_v14_pipeline_with_analytics, EnhancedAnalyticsResult, AnalyticsEnablement
`pipeline_v14_enhanced.py`	library	●	Production analytics orchestration; runs the full v14 finance pipeline (config load, strict validation, cashflow, debt enrichment, WACC/discount resolution, KPI assembly) and exposes it as run_v14_pipeline.	run_v14_pipeline_enhanced, run_v14_pipeline (alias), PipelineMetrics, PipelineValidationError, PipelineConfigError
`reproducible_workbook.py`	emitter	●	Post-processes a written .xlsx to be byte-reproducible by freezing docProps/core.xml timestamps and pinning the ZIP date_time on every member (and an object-level companion), so canon byte-identity tests stay stable; touches only metadata, no KPI cells.	normalize_xlsx_reproducible, freeze_workbook_reproducible
`run_manifest.py`	library	●	Builds an auditable, tamper-evident run manifest binding each pipeline output to its resolved config hash, engine VERSION, and git commit for lender/auditor reproducibility.	build_run_manifest, engine_version, config_sha256, git_sha, RunManifest
`run_modes.py`	library	●	Run-mode contract and policy table (#733 honesty gate) that declares a run's grade (screening/developer/lender) and what toy-fallback/MC-floor behavior each grade permits.	RunMode, RunModePolicy, resolve_run_mode
`scenario_analytics.py`	library	●	Library-only batch orchestrator that evaluates many scenarios through the v14 cashflow/debt modules, inferring DSCR/CFADS and emitting optional batch summaries/charts.	ScenarioAnalytics, BatchScenarioResult, BatchResultSummary
`scenario_loader.py`	library	●	Universal YAML/JSON scenario config loader with light structural checks and a strict FX resolver; loads base configs for the pipeline and analytics layers.	load_scenario_config, ScenarioConfigError

Module	Kind	Cov	Purpose	Key API
schema_guard.py	schema	●	Strict-by-default pre-flight config validator (CESSPIT gatekeeper): validates FX, technology types, run mode, indirect taxes, and grid conditionals with no silent defaults.	validate_config_for_v14, ConfigValidationError
sensitivity_pareto.py	legacy	●	Legacy backward-compat shim delegating optimize_from_sensitivity_insights (and re-exported Pareto types) to the canonical analytics.sensitivity.optimizer; holds no logic.	optimize_from_sensitivity_insights, run_pareto_search, ObjectiveSpec, ParameterGridSpec, ParetoResult, export_pareto_table (re-exported)
sensitivity_v14.py	legacy	●	Deprecated backward-compat shim that emits a DeprecationWarning and star-re-exports the canonical analytics.sensitivity.engine sensitivity API.	run_sensitivity_analysis, build_one_way_sensitivity_suite (re-exported from analytics.sensitivity.engine)
three_statement.py	library	●	Assembles articulating three-statement outputs (P&L / cash flow / balance sheet) in USD from the engine's enriched annual_rows and debt_result, with tie-out checks; additive and KPI-neutral.	build_three_statement, build_three_statement_from_run, build_cashflow_waterfall, ThreeStatementResult, IncomeStatementRow, CashFlowRow, BalanceSheetRow
wacc_sensitivity.py	library	●	Sweeps cost-of-equity (ke) across a band in build_up WACC mode via evaluate_with_overrides, recomputing the achieved discount rate and reporting project/equity NPV and IRR at each ke so the knife-edge NPV swing is explicit.	ke_band_npv

analytics/core (9 — ●7 ●2 ○0)

Module	Kind	Cov	Purpose	Key API
__init__.py	library	●	Package aggregator for analytics.core: re-exports the returns, risk-metrics, parameter-solver, and sensitivity-runner public APIs used across analytics.	re-exports returns/risk_metrics/parameter_solvers/sensitivity_runner symbols via **all**
covenant_breach.py	library	●	Noise-tolerant covenant-breach probability primitives (single source of truth): prob_breach and is_floor_pinned guard against sub-ULP DSCR-sculpt pin noise being miscounted as spurious breaches (#657/#725).	prob_breach, is_floor_pinned, PROB_BREACH_RTOL
epc_helper.py	library	●	Backward-compatibility shim re-exporting epc_breakdown_from_config and epc_breakdown_dict from finance.epc_helper_v14 under the analytics.core namespace.	epc_breakdown_from_config, epc_breakdown_dict
exceedance.py	library	●	Shared, dependency-free exceedance-probability primitives (P50/P75/P90 z-score table + P_x = P50*(1 - z*sigma_pct/100)) used by both the wind and solar bankable-AEP uncertainty build-ups.	EXCEEDANCE_Z, exceedance_value
metrics.py	library	●	Canonical v14 scenario-KPI aggregation engine: computes project NPV/IRR (via finance.irr), DSCR statistics, CFADS aggregates, and lender KPIs (avg_dscr, llcr, plcr, headline min_dscr) from config + annual rows + debt result.	calculate_scenario_kpis, compute_kpis, _summary_stats
parameter_solvers.py	library	●	On-demand reverse-engineering solvers (tariff/debt/capex from IRR/DSCR/NPV/LLCR targets) routing solely through analytics.evaluate_scenario.evaluate_with_overrides; standalone analyst tools with no committed-scenario caller.	solve_for_tariff_given_irr, solve_for_tariff_given_equity_irr, solve_for_tariff_given_npv, solve_for_max_debt_given_dscr, solve_for_max_debt_multi_covenant, solve_for_min_capex_given_irr_floor, solve_tariff_breakeven, get_solver, SOLVER_REGISTRY
returns.py	library	●	Project & equity returns analytics with Pydantic V2 contracts; NPV/IRR/MIRR delegate to finance.irr and the module computes project/equity return summaries (MOIC/DPI, etc.).	ReturnsConfig, ProjectReturns, EquityReturns, AllReturns, calculate_npv, calculate_irr, calculate_mirr, calculate_project_returns, calculate_equity_returns, summarize_all_returns
risk_metrics.py	library	●	Tail-risk analytics (VaR, CVaR, downside risk, covenant-breach analysis) with Pydantic V2 contracts; TailRiskAnalyzer produces lender risk summaries and routes breach probabilities through covenant_breach.prob_breach.	TailRiskAnalyzer, RiskConfig, VaRCVaRResult, PercentileAnalysis, DownsideRisk, CovenantBreachAnalysis, MetricRiskSummary, TailRiskReport
sensitivity_runner.py	library	●	Path-loading entrypoint for deterministic one-way tornado sensitivity: loads a base scenario, builds default one-way driver sweeps, and delegates evaluation/contract assembly to analytics.sensitivity.engine.	run_sensitivity_analysis (alias), run_sensitivity_analysis_from_path

analytics/contracts (1 — ●0 ●1 ○0)

Module	Kind	Cov	Purpose	Key API
__init__.py	contract	●		

Module	Kind	Cov	Purpose	Key API
			Thin public-API facade that re-exports the canonical v14 contract types from analytics.contracts_v14, preserving both `analytics.contracts` and `analytics.contracts_v14` import styles.	re-exports analytics.contracts_v14.* (star + **all**)

`analytics/mc` (9 — ●8 ●1 ○0)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	library	●	Package init for the canonical Monte Carlo engine; uses lazy **getattr** to expose the engine, correlation, exports and aggregation public API without triggering evaluation_v14 import-time circulars.	MonteCarloEngine, run_monte_carlo_analysis, CorrelationSpec, load_correlation_from_config, apply_correlation_structure, validate_correlation_matrix, CovenantSpec, build_lender_risk_table, build_casper_risk_blocks, aggregate_trials (all lazy via **getattr**)
`aggregate.py`	library	●	Aggregates per-trial KPI dicts into a MonteCarloResult: complete-case row-aligned trial arrays, per-metric summary stats, percentile lookup (default P1/5/10/50/90/95/99), and the flat legacy scalar surface plus toy-fallback/partial-trial provenance counts.	aggregate_trials, DEFAULT_PERCENTILES
`convergence.py`	library	●	Read-only post-hoc MC convergence diagnostics: running-mean + CI half-width trace vs trial count, and distribution-free order-statistic CIs for P90/P95 plus a Wilson CI for covenant-breach probability. Never changes n_trials or the reported bands.	convergence_diagnostic, percentile_ci_diagnostic, DEFAULT_CI_PERCENTILES, Z_95
`correlation.py`	library	●	Single source of truth for MC correlation: CorrelationSpec carrier, matrix validation/nearest-PSD repair, method dispatch (Iman-Conover rank reorder default, opt-in exact-normal-scores Gaussian copula), config loaders and active-param re-indexing.	CorrelationSpec, validate_correlation_matrix, apply_correlation_structure, load_correlation_from_config, align_correlation_to_params, apply_correlation_to_lhs, get_renewable_energy_correlation_template, SUPPORTED_CORRELATION_METHODS
`covenant.py`	library	●	Dependency-light (stdlib-only) shared DSCR covenant-floor resolution used by both the MC engine and the CASPER mc_risk block; config-first precedence over constraints/Financing_Terms/monte_carlo with a 1.30 default.	resolve_config_value, resolve_min_dscr_covenant, MIN_DSCR_COVENANT_PATHS, DEFAULT_MIN_DSCR_COVENANT
`degradation.py`	library	●	Optional per-trial degradation hook: when monte_carlo.degradation.enabled it injects the project.degradation override (respecting a sampled draw), else returns overrides unchanged.	apply_degradation_if_enabled
`engine.py`	library	●	Canonical Monte Carlo engine: samples the scenario's monte_carlo.parameters (LHS default, opt-in Sobol), applies distribution shapes/CF-coupling/FX-calibration/correlation/degradation, evaluates each trial through evaluation_v14, and returns an aggregated MonteCarloResult with VaR/CVaR, fixed-debt breach stress and convergence diagnostics.	MonteCarloEngine, run_monte_carlo_analysis, MonteCarloConfigError, MonteCarloRunMeta
`exports.py`	library	●	Thin, pure lender-style exports for a MonteCarloResult: a P50/P90/P95-downside risk table over DSCR/IRR/NPV/LLCR/PLCR, DSCR covenant-breach probability (floor-pinned aware) and worst-year DSCR P95, plus a CASPER-ready dict-of-tables payload.	CovenantSpec, build_lender_risk_table, build_casper_risk_blocks, dscr_breach_probability, worst_year_dscr_p95
`samplers.py`	library	●	Sampling utilities for the MC engine: the canonical stratified Latin Hypercube sampler (with a shared/independent permutation-stream flag) and an opt-in Sobol low-discrepancy QMC sampler (power-of-two, scipy.stats.qmc, lazily imported).	generate_lhs_samples, generate_sobol_samples

`analytics/sensitivity` (13 — ●0 ●12 ○1)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	library	●	Package init for the deterministic sensitivity suite; lazy **getattr** exposes the tornado engine, interaction grid, tail-risk and export helpers without an evaluation_v14 import-time circular.	SensitivityRunConfig, run_sensitivity_analysis, build_one_way_sensitivity_suite, build_two_factor_interaction_grid, TailRiskConfig, enrich_suite_with_tail_risk, suite_to_tables, suite_to_records (all lazy)
`adapters.py`	library	●	Bidirectional adapter between the generic sensitivity engine and contracts_v14 Pydantic models: converts ParameterRangeConfig to engine specs / sweep cases and engine case output back into TornadoResult / SensitivitySuite.	parameter_to_engine_spec, iter_param_cases_from_contract, engine_to_tornado_result, engine_to_sensitivity_suite
`dashboard_demo.py`	sandbox	○	Illustrative usage snippet (not a runnable module) showing how to build a SensitivityRequest, run a tornado, export a dataframe, plot a chart and enrich with tail risk / Pareto search.	

Module	Kind	Cov	Purpose	Key API
`docstrings.py`	library	●	Shared help-text/overview strings (sensitivity overview, tornado-chart doc, lender-view doc) reused across sensitivity modules for consistent terminology.	SENSITIVITY_OVERVIEW, TORNADO_CHART_DOC, LENDER_VIEW_DOC
`dscr.py`	library	●	Minimal DSCR-focused convenience wrapper over the one-way sensitivity engine (deterministic tornado of dscr_min for a single parameter); covenant/breach evaluation is delegated to tail-risk enrichment.	run_dscr_one_way, DscrSensitivityConfig
`engine.py`	library	●	Canonical orchestration hub for deterministic one-way/multi-parameter tornado sensitivity: validates driver paths, evaluates base + low/high shocks via the evaluation_v14 gateway, assembles SensitivitySuite/TornadoResult, and flags structurally-flat (covenant-pinned) metrics.	run_sensitivity_analysis, build_one_way_sensitivity_suite, SensitivityRunConfig, _resolves_in_config
`export.py`	library	●	Pure transforms turning a SensitivitySuite into JSON-friendly records or (optional-pandas) dict-of-tables for CSV/ Excel/JSON export of tornado rows.	suite_to_records, suite_to_tables
`global_sa.py`	library	●	Global variance-based / moment-independent sensitivity (SALib): Morris screening, Sobol S1/ST indices and PAWN KS indices over the scenario's monte_carlo.parameters, with flat-metric and NaN-poisoning guards; SALib is an optional call-time dependency.	run_sobol, run_morris, run_pawn, build_problem, GlobalSAProblem
`interaction.py`	library	●	Two-factor interaction grid: evaluates a KPI over the low/base/high cross-product of two drivers and computes the interaction surface (deviation from one-way additivity) with base-mismatch and failed-cell disclosures.	build_two_factor_interaction_grid
`optimizer.py`	library	●	Bounded Pareto-style multi-objective explorer over parameter grids: grid / LHS plans and an opt-in NSGA-II (pymoo) search, evaluating each point via the evaluation_v14 gateway and returning the non-dominated frontier.	run_pareto_search, pareto_frontier, export_pareto_table, ObjectiveSpec, ParameterGridSpec, ParetoResult, ParetoPoint, build_grid_plan, build_lhs_plan
`tail_risk.py`	library	●	Tail-risk enrichment for a one-way SensitivitySuite: attaches a 3-point tornado snapshot (base/downside/upside/ impact) and a per-metric aggregate for the CASPER consumer; also holds a separate, currently-unwired MC-array distributional path (VaR/CVaR/breach).	enrich_suite_with_tail_risk, TailRiskConfig
`tax.py`	library	●	Placeholder convenience wrapper for one-way sensitivity over a tax parameter (e.g. corporate rate) on a chosen metric, via the one-way sensitivity engine.	run_tax_one_way, TaxSensitivityConfig
`validation.py`	test-support	●	Parameter-range QA checker: sweeps each parameter's low/high pct points through evaluate_with_overrides and flags suspicious metric outputs (negative IRR, IRR>50%, evaluation errors) as a DataFrame for tests/CI.	validate_parameter_ranges

`analytics/wind` (12 — ●3 ●9 ○0)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	library	●	Package init for analytics.wind; re-exports the wind-to-finance integration public API (AEP loading, MC, config integration, pipeline).	load_aep_for_project, run_aep_monte_carlo, compute_aep_p_values, integrate_aep_into_config, derive_capacity_factor_from_aep, integrate_aep_pipeline, validate_turbine_specs, WIND_MODULES_AVAILABLE
`aep_summary_builder.py`	library	●	Regenerates/validates the AEP summary JSON from a scenario config in one call (analytic Weibull integral + density correction + loss stack), and asserts the three curve identifiers (store slug, manifest source_id, display model) agree — closing the curve-selection two-stage drift gap.	validate_curve_selection, build_aep_summary_from_config, write_aep_summary
`aep_tornado.py`	library	●	Deterministic one-at-a-time AEP tornado (wind-speed bias, shear, losses, power curve) via an analytic Weibull integral, ranked by swing for the lender tornado chart.	tornado_from_config, run_aep_tornado, gross_aep_farm_gwh, write_tornado_csv, AEPTornadoConfig
`crossval_interface_schema.py`	schema	●	Registers the opt-in resource.crossval config-schema (enabled flag + source_type merra2/local/newa) for second-source ERA5 cross-validation, enforced at pre-flight only when the block is declared.	CROSSVAL_INTERFACE_MODULE (register_required_fields side-effect on import)
`era5_interface_schema.py`	schema	●	Registers the resource.era5 config-schema (site centroid, reference-window mode, hub height, strict_coverage) that drives ARCO single-point ERA5 retrieval from the same scenario file, enforced via schema_guard module 'era5'.	ERA5_INTERFACE_MODULE, VALID_REFERENCE_MODES (register_required_fields side-effect on import)
`losses_model.py`	library	●	Applies the config-first, IEC 61400-15-2-aligned resource.losses stack multiplicatively (gross->net AEP), with a fail-loud taxonomy (reduction vs uptime) that refuses to silently drop an unknown loss key.	apply_losses, compute_net_factor, validate_loss_keys, net_capacity_factor, build_aep_losses_block, default_loss_taxonomy, LossResult, DEFAULT_LOSS_TAXONOMY, DEFAULT_WIND_LOSSES
`mc_aep_weibull.py`	library	●		fit_weibull_from_series, WeibullFit (+ MC runner/summary writers)

Module	Kind	Cov	Purpose	Key API
			Monte-Carlo AEP from an ECMWF-derived Weibull fit: samples plausible annual (A,k) resources, runs each synthetic year's net AEP through the OEM curve + loss stack, and reports P50/P75/P90/P99 exceedance percentiles.	
`pipeline_aep_v14.py`	library	●	Orchestrates the wind->AEP->revenue chain for v14: config-driven AEP-summary loading, turbine-spec consistency validation, and optional Monte-Carlo AEP integration.	integrate_aep_pipeline, validate_turbine_specs
`siting_metadata.py`	library	●	Resolves the optional resource.siting terrain_class into the AEP-summary provenance/diagnostics block as pure metadata (applies NO AEP correction); companion label to the single-cell representativeness diagnostic.	resolve_siting_metadata, TERRAIN_CLASSES, SITING_METADATA_NOTE
`wind_integration.py`	library	●	Wind-to-finance bridge: loads AEP for a project, injects AEP data into the finance config, derives capacity factor from AEP, and runs/extracts AEP Monte-Carlo p-values (graceful-degradation optional imports).	load_aep_for_project, integrate_aep_into_config, run_aep_monte_carlo, compute_aep_p_values, derive_capacity_factor_from_aep, WIND_MODULES_AVAILABLE
`wind_interface_schema.py`	schema	●	Registers the normalized resource.wind config-schema (ws150 mean/std, capacity_factor, aep_gwh, source_id/source_type) — the stable GIS->EPC handoff contract that cashflow/metrics depend on — enforced via schema_guard module 'wind'.	WIND_INTERFACE_MODULE, VALID_SOURCE_TYPES (register_required_fields side-effect on import)
`wind_rose.py`	library	●	Standalone directional wind-rose OUTPUT builder (bins met-convention directions into sectors, optional per-sector mean speed / energy rose / sectorwise Weibull) as display/provenance only — never scales AEP.	build_wind_rose, DEFAULT_CALM_LIMIT_MS, ROSE_PROVENANCE_NOTE

`analytics/power_curves` (1 — ●1 ●0 ○0)

Module	Kind	Cov	Purpose	Key API
`oem_parser.py`	library	●	Parses the config-sourced OEM/reference power curve from the canonical power_curves.yaml store and applies the IEC 61400-12-1 air-density (velocity-cube) correction; also computes AEP and IEC-compliance checks from a curve.	parse_power_curve, parse_envision_en171_curve, interpolate_power_curve, compute_aep_from_curve, validate_power_curve_iec_compliance, POWER_CURVE_STORE, CANONICAL_CURVE_KEY, ENVISION_EN171_65_POWER_CURVE, ENVISION_EN171_65_SPECS

`analytics/loader` (1 — ●1 ●0 ○0)

Module	Kind	Cov	Purpose	Key API
`aep_loader.py`	library	●	Loads pre-computed AEP summaries from the Data Lake with SHA-256 checksum, approved-source-manifest validation, and standardized provenance (source_id/IEC standard/placeholder-guard) for lender-grade AEP.	load_aep_from_summary, validate_source_manifest, register_approved_source, load_approved_sources_from_yaml, assert_source_in_manifest, is_placeholder_source, has_certified_oem_curve, build_provenance_aep_block, validate_config_aep_provenance, compute_checksum_sha256, create_aep_summary_template, export_provenance_report, APPROVED_SOURCES, IEC_STANDARDS

`analytics/gis` (9 — ●2 ●7 ○0)

Module	Kind	Cov	Purpose	Key API
`boundary_clip.py`	emitter	●	Clips a single-band wind/AEP GeoTIFF to a project-boundary polygon (issue #825, revives #21) via rasterio.mask, writing a clipped raster with a cos-latitude area-weighted in-polygon stats block plus provenance and an optional manifest entry. KPI-neutral reporting layer.	clip_to_polygon, clipped_domain_stats, load_polygon
`dem_ingest.py`	emitter	●	Loads a local Copernicus GLO-30 (SRTM fallback) DEM, clips to the site bbox and derives slope/aspect/hillshade via Horn's 3x3 kernel, exporting each as a provenance-stamped GeoTIFF plus a manifest entry; the slope layer feeds RIX (#831) and the siting mask (#833).	run_dem_ingest, load_dem, derive_terrain_layers, slope_aspect_hillshade, DEM_SOURCES, TERRAIN_VARIABLES, main
`exclusion_mask.py`	emitter	●		run_exclusion_mask, build_buildable_mask, build_exclusion_geometry, build_setback_zone, MASK_VARIABLE, main

Module	Kind	Cov	Purpose	Key API
			Builds a Boolean buildable-area siting mask (issue #833) from config-first setback buffers, protected-area/wetland polygons and a slope threshold, unioning exclusions and rasterising the complement to a provenance-stamped GeoTIFF plus a manifest entry. KPI-neutral.	
`geotiff_export.py`	library	●	Low-level GeoTIFF writer for the GIS stack: writes single-band float32 EPSG:4326 rasters with a north-up affine and provenance sidecars, and builds/appends DataLake_Manifest_All.json entries. rasterio is a CASPER-guarded optional [gis] dep.	write_geotiff, export_grid_rasters, build_manifest_entry, append_manifest, DEFAULT_CRS, BBox, _require_rasterio
`gis_export.py`	emitter	●	Orchestrates the issue-#20 GIS export driven by a scenario `gis:` block: samples ERA5 per-cell to build coarse WS150/CF/AEP grids, bilinearly downscales fine grids, and writes GeoTIFFs plus a DataLake manifest with a read-only spatial-representativeness verdict.	run_gis_export, grid_specs, default_era5_source, CellSource, main
`gwa_ingest.py`	emitter	◐	Ingests a locally supplied Global Wind Atlas 250 m GeoTIFF as a KPI-neutral reference/visualization layer: clips (and reprojects if needed) to the site bbox, then writes a provenance-stamped GeoTIFF plus a DataLake manifest entry.	run_gwa_ingest, ingest_gwa_layer, gwa_provenance
`landcover_roughness.py`	emitter	◐	Reclassifies an ESA WorldCover ~10 m land-cover GeoTIFF to an aerodynamic roughness-length (z0) raster via a cited WMO/Davenport-Wieringa lookup and exports a KPI-neutral provenance-stamped z0 GeoTIFF plus a manifest entry.	export_roughness_raster, reclass_landcover_to_z0, DEFAULT_LANDCOVER_Z0, DEFAULT_Z0_FALLBACK
`mcdm_suitability.py`	emitter	◐	GIS-MCDM wind-siting suitability surface (issue #834): normalises criterion rasters (wind/slope/roughness), derives AHP pairwise weights with a Consistency Ratio, combines by weighted linear combination and applies the Boolean exclusion mask as a hard veto, exporting a suitability GeoTIFF plus manifest. KPI-neutral.	build_suitability_surface, export_suitability_surface, normalise_criterion, ahp_priority_vector, weighted_linear_combination, apply_exclusion_veto, SAATY_RANDOM_INDEX
`rix.py`	library	◐	Computes the RIX terrain-ruggedness index and ΔRIX envelope disclosure (issue #831) from the DEM slope raster, flagging where linear-flow/reanalysis (WAsP/ERA5) assumptions need a terrain correction; pure-numpy diagnostic with an optional RIX raster export. KPI-neutral.	rix_at_point, delta_rix, assess_delta_rix, build_rix_report, rix_field, export_rix_raster, RixDisclosure, RixReport, critical_slope_ratio_to_degrees

`analytics/grid` (18 — ●0 ◐16 ○2)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	library	◐	Package init for the in-house design-stage grid interconnection / hosting-capacity study (SCR@POC, reactive, ride-through, harmonics, curtailment); documents that everything here is additive, default-OFF, and never imported by the finance engine.	
`__init__.py`	library	◐	Package init for the per-technology grid-capability plug-ins (wind/solar/BESS) that answer the converter-interface fault-current and PQ capability question keyed on finance.tech_types.	
`bess.py`	library	○	BESS storage grid-capability plug-in: feeds a manual per-site PCS short-circuit contribution into the SCR screen and screens SoH-degraded reactive/PQ capability (year-15 headroom shrinkage).	screen_bess_capability, bess_sc_contribution_pu, degrade_bess_rating, BessDegradedRating
`bess_soc.py`	library	○	Shared BESS state-of-charge model and reserve-split accounting: computes dischargeable/chargeable MWh at a given SoC and enforces the no-double-count energy invariant across competing reserves.	bess_soc_state, split_reserves
`solar.py`	library	◐	Solar PV grid-capability plug-in: models PV as a current-limited grid-following source (~1.1 pu fault current, must not be a synchronous machine) and derives DC:AC clipping and reactive PQ capability.	screen_solar_capability, solar_fault_contribution, ac_rating_from_dc, is_solar_capability_tech, SolarFaultContribution
`wind.py`	library	◐	Wind-turbine grid-capability plug-in: builds the reactive PQ envelope (Type-3 DFIG partial-load narrowing vs Type-4 full-converter rectangular box) and LVRT ride-through via the shared dynamics core.	screen_wind_capability, build_wind_pq_envelope, resolve_wind_converter_spec, q_capability_mvar_at_p, WindConverterSpec, WindCapabilityResult
`curtailment_qsts.py`	library	◐	OpenDSSDirect QSTS curtailment engine: runs a quasi-static time-series power-flow over a real feeder, injects per-tech hybrid profiles behind a POC export cap, and splits curtailed energy into deemed-paid (KPI-neutral) vs self-curtailed.	run_qsts_curtailment, split_curtailment
`evaluate_grid.py`	library	◐	CCCDIR single gateway for the grid study: composes the SCR strength screen and the reactive/voltage screen into one GridStudyResult; does no electrical work when grid.study_enabled is off.	evaluate_grid
`frequency_response.py`	library	◐	Closed-form frequency-response de-load / droop headroom sizer: computes the MW headroom and annual MWh a plant foregoes to meet a grid-code primary-frequency-response obligation.	size_frequency_response_headroom, freq_band_from_gridcode, droop_reserve_fraction, FrequencyResponseHeadroom
`grid_interface_schema.py`	schema	◐	Strict config-first schema/validation for the top-level grid block (study_enabled gate plus poc/thevenin/scr/gridcode/per-tech sub-blocks); registered into schema_guard only when a scenario declares a grid block.	validate_grid_block

Module	Kind	Cov	Purpose	Key API
harmonics.py	library	●	SCR-coupled power-quality screen at the POC: estimates harmonic voltage distortion vs IEEE 519:2022 limits and IEC 61400-21 flicker Pst as a function of grid stiffness (SCR).	screen_harmonics, ieee519_voltage_limits, ieee519_current_tdd_limit, flicker_pst, total_harmonic_distortion_pct, poc_ssc_mva, isc_il_ratio
__init__.py	library	●	Package init for static hybrid-POC aggregation (composite/weighted SCR and aggregate PQ envelope) for a fleet of wind+solar+BESS resources behind one point of connection.	
frequency_response.py	library	●	Combined ENPPC-style hybrid frequency-droop study: splits a frequency event's P(f) response across dispatch groups by priority order, caps each at its physical headroom, and screens the settling frequency against the grid-code band.	run_hybrid_frequency_response, compute_group_droop_split
poc_aggregation.py	library	●	Static hybrid-POC aggregation: walks the resolved multi-tech resources, folds each onto one POC bus, and emits the composite weighted SCR and aggregate reactive-capability envelope.	aggregate_poc_capability, resource_contribution
reactive_screen.py	library	●	Steady-state reactive/voltage PQ-box screen at the POC: runs a pandapower AC load-flow over a P x grid-voltage sweep and measures any Mvar shortfall against the grid-code PQ box.	screen_reactive_capability, pf_to_qp_ratio
ride_through.py	library	●	Shared ANDES RMS ride-through dynamics core running LVRT/HVRT/frequency grid-code cases against a generic WECC IBR, deriving pass/fail from the physical envelope (not raw solver convergence).	run_ride_through_case, run_ride_through_suite, envelope_from_fixture, build_case_spec, RideThroughEnvelope, RideThroughCaseSpec
ride_through_poc.py	legacy	●	Backward-compatibility SHIM for the original D1 LVRT scaffold; run_lvrt_case now delegates to the generalised ride_through core.	run_lvrt_case, lvrt_enter_from_fixture, LvrtScaffoldResult
short_circuit.py	library	●	Screens grid strength via SCR@POC = S_sc/S_plant using pandapower IEC 60909 (min/max) with a closed-form Thevenin fallback, banding the result into a GFL-vs-GFM viability screen.	screen_grid_strength

analytics/cost (6 — ●2 ●4 ○0)

Module	Kind	Cov	Purpose	Key API
__init__.py	library	●	Package marker/docstring for the cost-stack helpers (cost-basis year, contingency, estimate class, probabilistic CAPEX, benchmarks).	
benchmark.py	library	●	Advisory top-down cost sanity checks vs published anchors: capex_benchmark (project $/kW vs IRENA global onshore-wind TIC) and lcos_benchmark (BESS LCOS vs Lazard/PNNL band); report-only, never mutates values.	capex_benchmark, lcos_benchmark, irena_benchmark_per_kw
contingency.py	library	●	AACE RP 119R-21 CAPEX contingency: parametric QRA increment (base-cost uncertainty x z(confidence)) when capex.contingency.method=qra, else the fixed line-item fallback; config-first.	ContingencyResult, contingency_is_qra, resolve_contingency
cost_basis.py	library	●	Resolves the single anchored USD cost-basis year for the CAPEX/OPEX stack (NREL-ATB style), config-first: scenario capex.cost_basis_year else config/defaults.yaml cost_reference.basis_year.	default_cost_basis_year, resolve_cost_basis_year
estimate_class.py	library	●	First-class AACE estimate-class attribute (5..1) resolving the asymmetric low/high accuracy band from config; the band also floors the probabilistic-CAPEX 1-sigma; config-first (never a literal).	AccuracyBand, resolve_accuracy_band
mc_capex.py	library	●	Probabilistic CAPEX: samples total CAPEX ~ Normal(base, sigma) and re-runs the v14 pipeline per draw (dual-DSCR debt sizer re-solves) to return percentile economics; sigma floored to the AACE estimate-class band.	CapexMcResult, run_capex_mc, resolve_capex_sigma_pct

analytics/fx (7 — ●5 ●2 ○0)

Module	Kind	Cov	Purpose	Key API
__init__.py	library	●	FX package facade using PEP 562 lazy loading to break the contracts_v14 import cycle; lazily exposes integrate_fx_into_scenario_result on first access.	integrate_fx_into_scenario_result (lazy **getattr**)
fx_builder.py	library	●	Factory functions that build the FX structured block, FX curve time-series, and lender-grade FX risk profile (VaR/CVaR/concentration) from config + debt result + cashflow rows.	compute_fx_structured_block, compute_fx_curve, compute_fx_risk_profile
fx_calibration.py	library	●		calibrate_from_config, FXCalibration (calibration spec)

Module	Kind	Cov	Purpose	Key API
			Calibrates the Monte-Carlo USD/LKR FX driver from historical data into a drift + volatility + crisis-regime mixture spec, replacing the hand-set uniform bound.	
`fx_contracts.py`	contract	●	Canonical immutable FX contract dataclasses/models: FXStructuredBlock, FXCurveOutput, FXRiskProfile, FXVolumetry, wired onto ScenarioResult during pipeline execution.	FXStructuredBlock, FXCurveOutput, FXRiskProfile, FXVolumetry
`fx_fetch.py`	library	●	Config-driven USD/LKR spot-rate resolution — resolves a pinned FIXED vintage (default, offline) or fetches LATEST with provenance, plus validate_fx drift check; the single FX source of truth.	default_fx_lkr_per_usd, FXRequestConfig, validate_fx
`fx_history.py`	library	●	Provenance-bearing historical USD/LKR daily-series loader (pinned CSV+JSON sidecar with SHA-256 verify, or live BIS/FRED refresh) that feeds FX Monte-Carlo calibration.	load_pinned_history, fetch_live_history_bis, fetch_live_history_fred, validate_history_drift
`fx_integration.py`	library	●	Pipeline entry point that builds the FX structured block/curve/risk profile and wires them onto an existing ScenarioResult during pipeline_v14 execution; degrades gracefully.	integrate_fx_into_scenario_result

`analytics/casper` (3 — ●0 ●3 ○0)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	library	●	Package facade for the CASPER lender payload subsystem; re-exports the orchestrator and payload builder plus the frozen CASPER_CONTRACT_VERSION.	evaluate_with_casper_tail_risk_and_payload, build_casper_payload, CASPER_CONTRACT_VERSION
`casper_payload.py`	emitter	●	Flattens engine outputs (ScenarioResult, MonteCarloResult, SensitivitySuite, generation/WBS) into the slender JSON-safe casper_result_v1 lender payload, including the MC lender-risk table and DSCR-covenant block.	build_casper_payload, CASPER_CONTRACT_VERSION
`casper_v14.py`	library	●	Thin CASPER/GWTF orchestrator that runs evaluation_v14.evaluate_with_casper_tail_risk and translates the CasperResult into the JSON payload via build_casper_payload.	evaluate_with_casper_tail_risk_and_payload

`analytics/portfolio` (5 — ●4 ●1 ○0)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	library	●	Package init for the portfolio-level multi-technology layer; re-exports the generation aggregator and the per-technology (Epic 6.1) tornado public surfaces.	SUPPORTED_TECHNOLOGIES, aggregate_generation, build_multi_tech_from_run, build_tech_generation_profile, build_wind_generation_profile, resolve_tech_aep_kwh, technology_breakdown, run_multi_tech_tornado, impact_by_technology, discover_generation_technologies (re-exports)
`generation_aggregator.py`	library	●	Builds the multi-tech generation contracts (GenerationProfile / MultiTechGenerationResult / TechnologyBreakdown) from a real finance run, apportioning the run's combined CFADS across declared technologies by operating margin (else by AEP); additive, no finance recomputation.	resolve_tech_aep_kwh, resolve_wind_aep_kwh, build_tech_generation_profile, build_wind_generation_profile, aggregate_generation, technology_breakdown, build_multi_tech_from_run, SUPPORTED_TECHNOLOGIES
`multi_tech_tornado.py`	library	●	Per-technology tornado sensitivity (Epic 6.1) for hybrid scenarios: sweeps each generation tech's coupled capex/CF/degradation drivers (and each storage tech's revenue lever) through the evaluate_with_overrides gateway to rank which technology drives IRR/covenant volatility.	MultiTechTornadoBar, discover_generation_technologies, discover_storage_technologies, discover_non_generation_technologies, applicable_drivers, applicable_storage_drivers, build_coupled_override, build_storage_override, run_multi_tech_tornado, impact_by_technology, flat_metrics
`poi_curtailment.py`	library	●	Models shared point-of-interconnection (POI) curtailment for a hybrid (ARCH-5, #476): when combined instantaneous per-tech hourly injection exceeds a shared export limit the excess is physically curtailed; opt-in and disclosure-only.	estimate_poi_curtailment, resolve_shared_poi_curtailment
`tech_wbs.py`	library	●	Per-technology CAPEX/OPEX/cost-of-capital work-breakdown (ARCH-3, #475) reconciled against the debt engine's financed totals; disclosure-only, KPI-neutral, and the single source of per-tech OPEX resolution for the aggregator's margin split.	build_multi_tech_wbs, resolve_tech_opex_usd, DEFAULT_RECONCILE_TOLERANCE_PCT

analytics/cli (5 — ●0 ●4 ○1)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	cli	●	Package facade for the Hydra-based analytics CLI wrappers (Monte Carlo, sensitivity, capital-structure optimizers, plus the deprecated argparse sensitivity CLI); documents usage and lists the modules but exports no symbols to avoid name clashes.	**all** (module name list only)
`cli_capital_structure_optimize_hydra.py`	cli	●	Canonical Hydra CLI for the opt-in debt-tranche-mix and capex-contingency optimizers (#740); an on-demand analysis tool that never participates in run_v14_pipeline.	main (Hydra entrypoint)
`cli_monte_carlo_hydra.py`	cli	●	Canonical Hydra CLI wrapper for Monte Carlo analysis, wired to analytics.mc.engine.run_monte_carlo_analysis; reads the scenario's monte_carlo.parameters block and emits a JSON summary.	main (Hydra entrypoint)
`cli_sensitivity.py`	legacy	○	DEPRECATED argparse-based sensitivity CLI (GWTF R3 violation), slated for removal in Sprint 18; delegates to analytics.core.sensitivity_runner.run_sensitivity_analysis. Superseded by cli_sensitivity_hydra.py.	main (argparse entrypoint)
`cli_sensitivity_hydra.py`	cli	●	Canonical Hydra CLI wrapper for one-way tornado sensitivity analysis, wired to analytics.core.sensitivity_runner.run_sensitivity_analysis; emits a SensitivitySuite JSON payload.	main (Hydra entrypoint)

analytics/dashboard (1 — ●0 ●0 ○1)

Module	Kind	Cov	Purpose	Key API
`streamlit_app.py`	emitter	○	Interactive Streamlit tornado-sensitivity explorer that runs analytics.core.sensitivity_runner (the same engine as the FastAPI /run-tornado endpoint) and renders results via analytics.sensitivity.export.	module-level Streamlit script (no exported symbols)

analytics/simulation (1 — ●0 ●1 ○0)

Module	Kind	Cov	Purpose	Key API
`monte_carlo_aep.py`	library	●	100k-scenario Monte Carlo for AEP uncertainty quantification (Weibull parameter + loss-factor variation) producing P50/P75/P90/P99 percentiles and confidence intervals for lender-grade AEP risk.	run_monte_carlo_aep

analytics/pysam_sandbox (2 — ●0 ●0 ○2)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	sandbox	○	Optional PySAM integration sandbox exposing a graceful-degradation PySAMRunner proxy for AEP profile extraction; only loaded when config generation.engine=pysam.	PySAMRunner, PYSAM_AVAILABLE
`pysam_runner.py`	sandbox	○	Minimal PySAM Windpower wrapper that runs a single simulation and returns a 20-year annual kWh generation profile with simple degradation; a reader, not a financial calculator.	PySAMRunner (get_annual_profile)

wind_resource (17 — ●6 ●11 ○0)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	library	●	Package init for the wind resource assessment subsystem; re-exports ERA5Fetcher, WindAnalyzer, EnergyCalculator, WindPipeline and pins **version** to the repo VERSION file.	ERA5Fetcher, WindAnalyzer, EnergyCalculator, WindPipeline, **version**

Module	Kind	Cov	Purpose	Key API
`arco_assessment.py`	library	●	Wires the ARCO single-point ERA5 retrieval into the analytic AEP chain: fits a Weibull on the hub-height series and VALIDATEs (never overwrites) the declared lender Weibull baseline, reporting drift and implied AEP.	era5_config_from_scenario, build_arco_assessment, assess
`bankable_aep.py`	library	●	Lender-grade bankable AEP engine: IEC air-density correction, Weibull-integrated gross AEP, PyWake granular wake + TI sensitivity, and IEC 61400-15-2 P50/P75/P90 uncertainty build-up with config-driven loss stack.	density_velocity_factor, gross_aep_weibull, model_wake_loss, wake_loss_ti_sensitivity, gaussian_k_star, budget_from_mapping, UncertaintyBudget, exceedance_levels, interannual_variability_drift, non_wake_retention
`cashflow_adapter.py`	library	●	Pure-function adapter that maps a WindPipeline cashflow export into a v14 scenario dict under overwrite/fill_if_absent/validate_only modes, guarding CF/tariff/FX alignment with a drift tolerance.	wind_export_to_scenario_patch, WindCashflowExport, WindAdapterDriftError, AdapterMode, DEFAULT_TOLERANCE_PCT
`__init__.py`	config	●	Marker/init for the wind_resource.config package that holds the YAML config assets (era5_config, power_curves, locations).	
`crossval.py`	library	●	Opt-in VALIDATE-mode second-source (MERRA-2/local/NEWA) cross-validation of the ERA5 wind resource: fits a Weibull on an injected reference series and discloses drift/deviation vs the declared ERA5 baseline without mutating it.	crossval_settings, build_crossval_assessment, fetch_merra2_series, load_reference_series, summarise_deviation, CROSSVAL_SOURCES
`energy_calculator.py`	library	●	Timeseries-integration AEP path: integrates a turbine power curve over the raw hub-height wind timeseries to gross AEP, applies the loss stack once for net AEP, and derives P50/P75/P90 and revenue for the wind_resource diagnostic pipeline.	EnergyCalculator
`era5_fetcher.py`	library	●	Legacy gridded ERA5 downloader via the Copernicus CDS API (10m/100m winds, hub-height power-law extrapolation, local caching) exposing the ERA5Fetcher class used by WindPipeline.	ERA5Fetcher
`era5_grid.py`	library	●	Builds coarse (native ~0.25°) and fine (bilinear-downscaled ~0.05°) n×n grids of ws150_mean/ capacity_factor/aep_per_turbine over a site for GIS export, sampling the ERA5 point pipeline per cell.	GridSpec, CellResult, assemble_grids, spatial_representativeness, downscale_bilinear, fetch_cell_results, GRID_VARIABLES
`era5_retrieval.py`	library	●	Config-driven ARCO/Zarr single-point ERA5 retrieval: fetches the CDS timeseries product, builds a clean hourly hub-height wind series, validates leap-aware coverage, computes site AEP, and builds the production wind rose used in the report.	ERA5RequestConfig, retrieve_era5_timeseries, build_hub_height_series, validate_coverage, compute_site_aep, build_production_wind_rose, run, main, ERA5CoverageError, expected_hours_for_years
`layout_optimizer.py`	library	●	Area→optimized-layout micro-siting tool: runs the DTU TopFarm gradient optimizer on PyWake against a boundary/exclusion/spacing/wind-rose spec to propose an AEP-maximising candidate layout and its uplift vs a baseline.	optimize_layout, TurbineSpec, LayoutOptimizationResult
`long_term_trend.py`	library	●	Long-term wind resource & trend analysis (Mann-Kendall + Sen's slope) over candidate reference periods, classifying decadal variability vs stilling and recommending the forward-looking P50 basis for the bankability report.	compute_trend, annual_mean_series, reference_periods, period_aep_table, recommend_p50, render_trend_markdown, analyze_long_term_resource, build_resource_trend_export_block, TrendResult
`mcp.py`	library	●	Measure-Correlate-Predict for long-term on-site wind resource: fits a variance-ratio/OLS transfer from a short on-site mast record against concurrent ERA5 and predicts the long-term on-site distribution (opt-in, VALIDATE only).	run_mcp, mcp_settings, variance_ratio_transfer, linear_regression_transfer, predict_long_term, pearson_r, MCPResult, MCP_METHODS
`power_curve_sourcing.py`	library	●	Power-curve sourcing/ingest for any turbine: fetch from the open oedb library (windpowerlib) or manual/OEM/WAsP-WTG/tabular entry, validating and stamping provenance before adding to the power_curves store.	fetch_oedb_power_curve, manual_power_curve, validate_power_curve, add_curve_to_store, from_wasp_wtg, from_tabular_file, fetch_turbine_models_curve, list_turbine_models, PowerCurve
`weibull_fit.py`	library	●	Fits a 2-parameter Weibull (scipy weibull_min MLE, floc=0) to a wind-speed series with goodness-of-fit metrics and reports drift vs a declared (A,k) baseline; the bridge for the ARCO/crossval/MCP validate paths.	fit_weibull_on_series, weibull_drift, weibull_std, energy_moment_gof_pct, WeibullFit, WeibullDrift
`wind_analyzer.py`	library	●	Wind resource statistical analysis: Weibull fitting, temporal (monthly/seasonal/diurnal) patterns, and inter-annual variability metrics for the WindPipeline diagnostic workflow.	WindAnalyzer
`wind_pipeline.py`	library	●	Orchestrates the full CSV/ERA5Fetcher wind assessment workflow (fetch → WindAnalyzer → EnergyCalculator → JSON export) and produces the export consumed by the v14 cashflow adapter.	WindPipeline

solar_resource (9 — ●1 ●8 ○0)

Module	Kind	Cov	Purpose	Key API
__init__.py	library	●	Package init for the optional [solar] PV producer subsystem; eagerly re-exports the pvlib producer, the pvlib-free cashflow adapter, the exceedance/loss/soiling/source-quality/bifacial provenance surfaces, and the GHI long-term trend.	SolarResourceConfig, SolarAEPResult, compute_solar_aep, validate_declared_solar_cf, build_solar_cashflow_export, solar_export_to_scenario_patch, exceedance_levels_solar, compute_net_solar_loss_factor, grade_solar_source_quality, assert_monofacial_financed_cf, soiling_profile_from_config (re-exports)
bifacial_guard.py	library	●	SOLAR-9 discipline guard: refuses a bifacial rear-side uplift from entering the FINANCED/ OVERWRITE solar CF chain (the committed pack finances the monofacial pvlib yield); fails loud rather than silently inflating the billed CF.	BifacialInFinancedChainError, detect_bifacial_uplift, assert_monofacial_financed_cf, BIFACIAL_MARKER_KEYS
cashflow_adapter.py	library	●	The OVERWRITE finance bridge: applies a frozen solar AEP export into a v14 hybrid scenario by patching the per-tech generation block and re-blending project.capacity_factor; pure over dicts (never imports pvlib).	SolarCashflowExport, build_solar_cashflow_export, solar_export_to_scenario_patch, SolarAdapterDriftError, DEFAULT_TOLERANCE_PCT
exceedance.py	library	●	Pure (pvlib-free) PV P50/P75/P90 exceedance build-up from a config-first 1-sigma uncertainty budget (IEC 61724-1 / IEA-PVPS Task 13), sharing the analytics.core.exceedance z-table with the wind side.	SolarUncertaintyBudget, SolarExceedanceResult, exceedance_levels_solar, solar_uncertainty_from_config
long_term_trend.py	library	●	Solar-GHI long-term trend analysis (Mann-Kendall + Sen's slope on a frozen multi-decade PVGIS SARAH-3 annual-GHI series) that recommends a forward P50 GHI basis and renders a report section / lender-workbook ResourceTrend sheet; report-only, pvlib-free.	SolarTrendResult, compute_solar_trend, analyze_long_term_solar_resource, build_solar_resource_trend_export_block, render_trend_markdown, trend_summary_dataframe, reference_periods, period_ghi_table, recommend_p50
loss_model.py	library	●	Itemised gross->net solar loss chain (config-first IEC 61724-1 / PVsyst taxonomy) that decomposes the flat system_loss_pct into named components, reusing the technology-neutral retention engine from analytics.wind.losses_model.	compute_net_solar_loss_factor, default_solar_loss_taxonomy, validate_solar_loss_keys, DEFAULT_SOLAR_LOSS_TAXONOMY
pv_producer.py	library	●	The pvlib-based solar AEP producer (optional [solar] extra): turns a fixed-tilt PV spec + measured GHI (or a frozen hourly TMY) into a bankable annual energy / capacity factor; VALIDATE-only against a declared P50 (overwrite lives in the adapter).	SolarResourceConfig, SolarAEPResult, SolarCfValidation, compute_solar_aep, validate_declared_solar_cf
soiling_profile.py	library	●	Optional time-varying soiling profile (accumulation rate + wash cadence) reduced to a single effective flat soiling percent for the producer's loss chain; default-off and byte-identical to flat soiling when absent.	SoilingProfile, compute_soiling_profile_effective_pct, soiling_profile_from_config
source_quality.py	library	●	Grades the resource evidence behind a modelled solar P50 (measurement type / years / dataset) into an A-D quality score for the solar_resource provenance block; pure metadata, wired to no billed field.	SolarSourceQuality, grade_solar_source_quality, solar_source_quality_from_config, MEASUREMENT_TYPE_SCORES, TMY_SOURCE_SCORES, AXIS_WEIGHTS

app (1 — ●0 ●0 ○1)

Module	Kind	Cov	Purpose	Key API
__init__.py	bootstrap	○	Package docstring for the framework-agnostic web/service seam between callers and the canonical v14 pipeline; no finance logic.	

app/reports (9 — ●6 ●3 ○0)

Module	Kind	Cov	Purpose	Key API
__init__.py	bootstrap	●	Lender-report package aggregating the presentation layer (config loader, ReportContext builder, HTML/PDF renderer) that projects a CaseResult into a report; no finance logic.	ReportConfig, load_report_config, ReportContext, build_report_context, render_report_html, render_report_pdf, ReportDependencyError
capital_risk_emit.py	emitter	●	Opt-in batch emitter that runs the canonical Monte-Carlo engine (analytics.mc, LHS + Iman-Conover), assembles a CapitalRiskReport, and renders a lender HTML report including the capital-risk (VaR/CVaR/breach-prob) section.	emit_capital_risk_report_from_pipeline

Module	Kind	Cov	Purpose	Key API
`grid_screening_emit.py`	emitter	●	Opt-in batch emitter that surfaces the in-house Python grid study (D1–D7 screens) in a lender report wrapped in an un-suppressible SCREENING-NOT-BANKABLE/EMT-gap caveat; advisory-only, moves no committed KPI.	emit_grid_screening_report, MANDATORY_SCREENING_CAVEAT
`interaction_grid_emit.py`	emitter	●	Opt-in batch emitter that surfaces a two-factor sensitivity interaction grid (NxM full-pipeline evaluations via analytics.sensitivity.interaction) in the lender report; config-required driver pair, fail-loud.	emit_interaction_grid_report
`renderer.py`	emitter	●	Renders a ReportContext to HTML via Jinja2 (core dep, unit-testable) and to PDF via a lazily-imported optional WeasyPrint, raising ReportDependencyError when the PDF backend is absent.	render_report_html, render_report_pdf, ReportDependencyError
`report_config.py`	config	●	Loads config/report_defaults.yaml into typed Pydantic models (branding, covenant thresholds, KPI display spec, risk register taxonomy); validation/exposure only, no finance logic.	ReportConfig, load_report_config, DEFAULT_CONFIG_PATH
`report_model.py`	emitter	●	Projects a CaseResult into a render-ready, frozen ReportContext — formatting KPIs per the display spec, flagging them against lender covenants, and building CP/readiness/evidence/feasibility sections; pure presentation.	build_report_context, ReportContext, CapitalRiskBlock, TornadoBlock, GlobalSABlock, fmt_pct/fmt_usd/fmt_x helpers
`tech_comparison_emit.py`	emitter	●	Opt-in batch emitter that runs multiple scenario configs through the canonical run_v14_pipeline and emits a side-by-side headline-KPI comparison table (wind lender case vs hybrid/solar) reconciling exactly to the committed KPIs.	emit_tech_comparison_report
`wind_rose_plot.py`	emitter	●	Display-only polar wind-rose renderer: projects analytics.wind.wind_rose.build_wind_rose output into a base64 data:image/png embed for the report; call-time matplotlib guard, fails soft to None.	render_wind_rose_polar_data_uri

`app/api` (7 — ●0 ●0 ○7)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	○	Package marker for the unified FastAPI surface (versioned /v1 routers + wizard POST /v1/cases).	
`auth.py`	library	○	Stdlib-only OAuth2 bearer authentication (hand-rolled HS256 JWT + PBKDF2 password hashing) gating the /cases and /jobs surfaces, with fail-closed production posture read from the environment.	get_current_subject, login_for_access_token, decode_token, hash_password, verify_password
`config.py`	config	○	Env-overridable operational limits (sync-route timeout, max concurrency) for the synchronous /cases* compute routes; deployment knobs, not finance inputs.	SYNC_ROUTE_TIMEOUT_SECONDS, MAX_CONCURRENT_SYNC_ROUTES (module constants)
`jobs_router.py`	emitter	○	FastAPI router for the async live-ERA5 job path (POST /jobs enqueue, GET /jobs/{id} poll, /jobs/{id}/events SSE); dispatches run_wind_job via BackgroundTasks or the arq/redis queue.	router, enqueue_job, get_job, job_events, get_store
`main.py`	entrypoint	○	Unified FastAPI application composing all routers under /v1 and adding the wizard-facing POST /v1/cases (+ /surface, /report.html|pdf|xlsx) endpoints; the uvicorn entrypoint (app.api.main:app).	app (FastAPI), build_case_surface, run_finance_case wiring, run_case_report_* endpoints
`responses.py`	schema	○	Typed Pydantic response models (CaseResult, API_CONTRACT_VERSION) that pin the client-facing HTTP response shape for the /v1/cases wizard surface.	CaseResult, API_CONTRACT_VERSION
`surface.py`	schema	○	Typed result-surface response models (KpiCard, TornadoChart, GlobalSaChart, CapitalRiskSurface, ArtifactLinks) projecting a ReportContext into the chart-ready payload the wizard client consumes (#844).	CaseSurface, KpiCard, TornadoChart, GlobalSaChart, CapitalRiskSurface, ArtifactLinks

`app/jobs` (8 — ●0 ●0 ○8)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	○	Package aggregating the async live-ERA5 job path exports (models, stores, runner, SSE); deliberately does not re-export the arq worker.	JobRecord, JobState, WindJobRequest, InMemoryJobStore, JobStore, RedisJobStore, run_wind_job, job_event_stream
`config.py`	config	○	Env-overridable operational knobs for the async job path (record retention, Redis TTL, SSE stream lifetime, jobs backend selection); deployment limits, not finance inputs.	MAX_RETAINED_JOBS, JOB_TTL_SECONDS, SSE_MAX_POLLS, JOBS_BACKEND, JOBS_QUEUE, JOBS_REDIS_URL
`models.py`	schema	○	Pydantic job-domain models (JobState lifecycle, JobRecord, JobProgress, WindJobRequest) describing an async job's request, state, and progress; carries a WindFarmInputs payload.	JobState, JobRecord, JobProgress, WindJobRequest, utc_now_iso, TERMINAL_STATES

Module	Kind	Cov	Purpose	Key API
`redis_store.py`	library	○	Durable cross-process JobStore backed by an injected synchronous Redis client (optional [jobs] extra), serialising job records as namespaced JSON so the arq worker and API share state.	RedisJobStore, RedisLike
`runner.py`	library	○	Framework-agnostic async job orchestration: drives the ERA5-assessment→finance chain, records every transition on the JobStore, and reports coarse progress; the slow ERA5 step is injected for testability.	run_wind_job, default_assessment, new_queued_record, TOTAL_STEPS
`sse.py`	library	○	Server-Sent Events progress stream for an async job: polls the JobStore and emits a frame on each state/progress change until a terminal frame; sleep is injectable for deterministic tests.	job_event_stream, format_sse
`store.py`	library	○	Job-record store abstraction (JobStore protocol) with the in-process InMemoryJobStore default; the persistence seam the runner, API, and SSE stream depend on.	JobStore, InMemoryJobStore
`worker.py`	entrypoint	○	Optional arq + Redis worker running the same run_wind_job orchestration under an arq CLI for durable, horizontally-scalable async jobs; loaded only by the arq CLI, requires the [jobs] extra.	WorkerSettings, run_wind_job_task

`app/services` (4 — ●0 ●0 ○4)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	○	Package re-exporting the in-memory service entry points (run_finance_case, run_integrated_case) over the canonical v14 pipeline.	run_finance_case, run_integrated_case, DEFAULT_VALIDATION_MODULES
`pipeline_service.py`	library	○	Framework-agnostic service seam wrapping the canonical run_v14_pipeline (and wind/solar cashflow adapters) with no file I/O or printing; the single backend entry for web/API/notebook callers.	run_finance_case, run_integrated_case, DEFAULT_VALIDATION_MODULES
`report_global_sa.py`	library	○	Computes the report's global sensitivity screening (Morris elementary effects, PAWN) by writing the in-memory scenario to a temp file and running the canonical analytics.sensitivity.global_sa engine; best-effort, returns None on failure.	compute_report_global_sa (Morris/PAWN block builder)
`report_tornado.py`	library	○	Computes the report's one-at-a-time sensitivity tornado by writing the in-memory scenario to a temp file and running the canonical sensitivity_runner sweep; best-effort, returns None on failure.	compute_report_tornado, TornadoRow

`app/web` (3 — ●0 ●0 ○3)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	○	Package docstring for the server-rendered HTMX wizard frontend (#843 backend-for-frontend) served inside the FastAPI app; describes auth_cookie and routes.	
`auth_cookie.py`	library	○	Cookie-based session gate for the HTMX wizard: stores the HS256 JWT in an httpOnly cookie and validates it by reusing app.api.auth.decode_token; owns only cookie transport, redirects to /login on failure.	require_web_user, set_session_cookie, clear_session_cookie
`routes.py`	emitter	○	HTMX wizard FastAPI router mounted at the app root: cookie login/logout flow and the four-step wind-farm case wizard whose Run/download actions reuse the /v1 in-process functions; parses forms stdlib-only (no python-multipart).	router (login, logout, wizard, wizard/run, wizard/report.{html,pdf,xlsx})

`app/models` (2 — ●0 ●0 ○2)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	○	Package re-exporting the wizard input models (ScenarioVariant, WindFarmInputs) for the web boundary.	ScenarioVariant, WindFarmInputs
`inputs.py`	schema	○	Customer-facing wizard input model (WindFarmInputs) that maps friendly form fields onto a committed base scenario and deep-merges them into a full v14 scenario dict via to_scenario_config(); a mapping layer only.	WindFarmInputs, ScenarioVariant, WindFarmInputs.to_scenario_config

api (4 — ●0 ●0 ○4)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	bootstrap	○	Package marker for the top-level HTTP API package (FastAPI adapters over analytics/finance).	
`path_safety.py`	library	○	Confines caller-supplied scenario/AEP file paths for the /run-pipeline and /run-tornado HTTP endpoints to allowed repo roots (path-traversal defence).	confined_path, allowed_roots, UnsafePathError
`pipeline_api.py`	emitter	○	FastAPI router (POST /run-pipeline) that runs the canonical run_v14_pipeline over an inline/path scenario and serialises headline KPIs, sculpted debt schedule, and bankable AEP for a lender/customer report.	router, run_pipeline endpoint, PipelineRequest models
`sensitivity_api.py`	entrypoint	○	FastAPI app + router for single-metric tornado sensitivity; delegates to analytics.core.sensitivity_runner and mounts the pipeline router; the top-level API entry (uvicorn).	app (FastAPI), SensitivityInput, SensitivityTornadoRow

scripts (34 — ●0 ●7 ○27)

Module	Kind	Cov	Purpose	Key API
`01_github_repo_scanner.py`	cli	○	Hydra-configured repository scanner producing inventory, dependency graph, tech-debt metrics, and migration-readiness reports.	main (hydra, conf/scanner.yaml)
`FINAL_CORRECTED_sensitivity_v14.py`	sandbox	○	Placeholder/patch-note stub referring a one-line breakeven bugfix out to a separate patch file rather than holding runnable logic.	
`FIX_SENSITIVITY_LINE_489.py`	sandbox	○	Documentation stub showing the exact replacement code for a TornadoResult construction bug near line 489 of the old sensitivity_v14.	
`add_pydantic_v2_compat_stubs.py`	legacy	○	One-off migration script that injects Pydantic v2 backward-compat stub models into analytics/contracts_v14.py to unblock legacy test imports.	main (source patcher)
`check_fields.py`	sandbox	○	Quick diagnostic dumping all field names present in the first annual_rows entry of a saved lendercase results JSON.	
`codebase_datalake_ingress.py`	cli	○	Repo-local codebase indexer producing structured CSV indexes of files and Python imports/exports/calls plus a relevance closure from analytics/finance.	main / indexer functions
`compile_changelog.py`	cli	○	Compiles per-PR changelog.d/ fragment files into the CHANGELOG.md [Unreleased] section (towncrier/scriv pattern) to avoid merge conflicts.	main / fragment-compiler
`datalake_refresh_and_diff.py`	cli	○	Recreates the ./datalake codebase snapshot artifacts in the repo root and diffs against the prior snapshot.	main / snapshot+diff routines
`dutchbay_cleanup_analyzer.py`	cli	○	Pre-cleanup analyzer that scans the repo and generates an inventory JSON of files that are candidates for cleanup.	main (--output report.json)
`dutchbay_manifest_builder.py`	cli	○	Builds a comprehensive manifest of v14-relevant scripts, docs, tests, and configs across the repo.	main (--output manifest.json)
`export_to_excel.py`	emitter	●	Standalone openpyxl exporter turning a finance result contract into a multi-sheet Excel workbook (annual cashflow, debt detail, covenants, KPI summary).	main / export functions (openpyxl)
`fix_metrics_typing.py`	legacy	○	One-off source patcher that rewrites analytics/core/metrics.py to make capex-derivation mypy-safe.	main (source patcher)
`fix_yaml_depreciation_method.py`	legacy	○	One-off migration that scans scenarios/ and tests/ YAMLs and injects a missing depreciation_method field into the tax section.	main (YAML patcher)
`gen_architecture_diagram.py`	cli	○	Generates docs/architecture_import_graph.mmd by AST-walking every first-party package and recording import edges so the diagram cannot drift from the code.	main / AST import-graph builder
`generate_solar_assessment_report.py`	emitter	○	Renders a lender-grade single-technology solar PV PDF assessment, baking together the solar resource producer, deterministic finance, LHS Monte Carlo, tornado, optimization, and a GIS location map.	main / report renderer
`go_with_the_flow_ci.py`	cli	○	Typer-based local CI orchestrator running black/isort/ruff/mypy/pytest with fire-fighting and artifact generation.	Typer app / CI commands
`inspect_era5.py`	sandbox	○	One-off diagnostic that opens a specific ERA5 NetCDF file and prints its dimensions, coordinates, and structure.	
`instrument_statutory_debug.py`	sandbox	○	Developer helper that searches analytics/finance for statutory-deduction tokens and (dry-run) reports or instruments them with debug statements.	main (--dry-run)
`make_clean_zip.py`	cli	○	Creates a clean, annotated archive (zip) of the repo for sharing/releases.	main / archive builder
`patch_contracts_v14_add_tornado_single.py`	legacy	○	One-off source patcher adding single-parameter TornadoResult and enhanced ParameterRangeConfig contracts to analytics/contracts_v14.py.	main (source patcher)

Module	Kind	Cov	Purpose	Key API
`process_era5_wind_data.py`	legacy	○	Sprint-10 ad-hoc script extracting wind speed from an ERA5 NetCDF and extrapolating per-turbine coordinates to 150 m hub height.	module-level extraction/ extrapolation routines
`provision_web_secrets.py`	cli	○	Mints production secrets (JWT signing key etc.) for the DutchBay web surface auth gate, printed for pasting into fly secrets set.	main / secret minter
`quarantine_bad_irr_mc_tests.py`	test-support	○	Scans and quarantines tests that violate current v14/MC contracts (hardcoded IRR bands, old top-level project_irr, deprecated MC signatures).	main / test-quarantine routine
`run_epc_margin.py`	cli	○	Thin CLI around finance.epc_margin.compute_epc_margin printing construction-margin economics (total cost, gross margin, return-on-cost, peak working capital, construction IRR) for an epc-block scenario.	main (argparse)
`run_full_pipeline_sprint12.py`	legacy	○	Sprint-12-era full-pipeline orchestrator (refinancing, equity distribution, Monte Carlo, stress tests) using a plain --config argparse CLI; superseded by run_full_pipeline_v14.py.	main (argparse)
`run_fx_calibration.py`	cli	●	CLI to inspect/validate the market-calibrated USD/LKR FX driver (drift, vol, regimes, spot percentiles) from the pinned historical vintage, with optional live-validation/refresh.	main (argparse)
`run_fx_sensitivity.py`	cli	●	Thin argparse CLI wrapping analytics.fx_sensitivity_real.FXSensitivityAnalyzer to sweep the three live FX levers (rate, hedge_ratio, spread_bps) and emit a single JSON sensitivity object.	main (argparse)
`run_global_sensitivity.py`	cli	●	Thin CLI around analytics.sensitivity.global_sa running SALib Morris screening or Sobol first/total-order indices over the monte_carlo.parameters drivers via the evaluate_with_overrides gateway.	main (argparse)
`run_multi_tech_tornado.py`	cli	●	Thin CLI around analytics.portfolio.multi_tech_tornado running a per-technology, coupled-override tornado for a hybrid multi-tech scenario and writing a flat CSV plus a stderr volatility ranking.	main (argparse)
`run_tornado_from_cli.py`	cli	●	Thin CLI around the canonical sensitivity engine (analytics.core.sensitivity_runner) that loads a scenario plus a YAML list of one-way sweeps and writes a flat tornado CSV per KPI metric.	main (argparse)
`run_wind_analysis_v14.py`	entrypoint	●	Hydra CLI for the complete wind-resource assessment (ERA5 download, Weibull stats, gross/net AEP P50/P75/P90, JSON export for the cashflow model).	main (hydra entrypoint)
`validate_pysam_offline.py`	sandbox	○	Offline PySAM validation comparing PySAM runner output against the legacy cashflow_v14 generation formula with a <5% deviation gate (no pipeline integration).	main (exit-code validator)
`verify_complete.py`	sandbox	○	One-off verification script that reads a saved lendercase results JSON and prints a DSCR / debt-service / CFADS table to confirm a specific DSCR fix.	
`verify_dscr.py`	sandbox	○	One-off script reading a saved lendercase results JSON and printing DSCR and debt-service by year for manual checking.	

`scripts/ci` (6 — ●0 ●0 ○6)

Module	Kind	Cov	Purpose	Key API
`check_all_py_files.py`	cli	○	CI helper that mypy-checks all Python files, filtering ignorable warnings.	check main
`check_legacy_imports.py`	cli	○	CI guard enforcing that no production code imports from legacy/ or dutchbay_v14chat/.	FORBIDDEN_PATTERNS, check main
`check_staged_py_files.py`	cli	○	Pre-commit/CI helper that mypy-checks only staged Python files, filtering ignorable warnings.	check main
`ci_structure_check.py`	cli	○	CI structure check validating directory layout, forbidden legacy imports, and absence of orphaned root files.	structure-check main
`model_guard.py`	cli	○	Quick CLI validating all parameter ranges for a scenario, detecting negative IRR, extreme swings, and missing values.	model-guard main
`validate.py`	legacy	○	Legacy v13-era config/YAML validation helper with graceful degradation when PyYAML is absent.	validation functions

scripts/build (3 — ●0 ●0 ○3)

Module	Kind	Cov	Purpose	Key API
build_zip_from_manifest.py	cli	○	Builds a deterministic-ish zip bundle from a JSON manifest with optional excludes and rename rules.	build/zip functions
generate_manifest.py	cli	○	Generates a JSON manifest describing the contents of an existing zip file.	create_manifest_from_zip
make_essential_zip.py	cli	○	Creates a zip of only essential code/config files, skipping outputs and legacy directories.	iter_essential_files, zip builder

scripts/github (2 — ●0 ●0 ○2)

Module	Kind	Cov	Purpose	Key API
auto_close_issues.py	cli	○	Auto-closes resolved GitHub issues via the gh CLI.	issue-close main
gh_tools.py	cli	○	Small helper CLI wrapping git + gh automation subcommands for the repo.	gh-tools subcommands

scripts/research (4 — ●0 ●0 ○4)

Module	Kind	Cov	Purpose	Key API
__init__.py	legacy	○	Package init for the standalone research-grade V12 multi-objective capital-structure optimiser (self-contained, independent of the live v14 engine).	
charts.py	legacy	○	Headless matplotlib chart helpers (e.g. tornado_chart) for the V12 Pareto capital-structure studies.	tornado_chart (and other chart helpers)
legacy_v12.py	legacy	○	The standalone Dutch Bay 150MW V12 financial model (self-contained cashflow/debt/IRR, CAPEX $155M wind-only) that predates and is independent of the live v14 engine.	V12 model functions
optimization.py	legacy	○	Multi-objective optimiser over debt ratio, USD/LKR split, and DFI debt under IRR/DSCR constraints for the standalone V12 model.	optimizer main

scripts/analysis (3 — ●0 ●0 ○3)

Module	Kind	Cov	Purpose	Key API
analyze_directory.py	cli	○	Directory-structure analyzer that traverses a root folder and emits a JSON view of the folder/file hierarchy and sizes.	main / traversal routine
gen_scenario_yaml.py	cli	○	Interactive prompt-driven generator that builds a scenario YAML from user-entered numeric inputs.	prompt_float, prompt_int, generator main
wacc_engine_yaml.py	legacy	○	Standalone YAML-driven WACC / hurdle-rate calculator; DEPRECATED, its build-up WACC methodology now lives in the live engine.	WACC calculator main

scripts/legacy_runners (2 — ●0 ●0 ○2)

Module	Kind	Cov	Purpose	Key API
run_complete_analysis_fixed.py	legacy	○	Legacy complete-analysis runner (cashflow, debt, older module set) predating the v14 pipeline.	runner main
run_wind_download_v14.py	legacy	○	Legacy Hydra CLI downloading ERA5 wind data from Copernicus CDS and extrapolating to hub height.	main (hydra entrypoint)

(repo root) (5 — ●2 ●1 ○2)

Module	Kind	Cov	Purpose	Key API
constants.py	config	●	Project-wide immutable physical constants and unit conversions only (hours/year, kW-MW, percent-decimal); all configurable defaults live in config/scenarios per config-first ARCH-01.	HOURS_PER_YEAR, DAYS_PER_YEAR, KW_TO_MW, MWH_TO_GWH (and other Final constants)
dutchbay_bootstrap.py	bootstrap	○	Read-only developer bootstrap helper: infers repo root, detects the shared .venv, and validates the GWTF ruleset environment; prints import-time diagnostics.	module-level bootstrap functions (repo-root/venv/ruleset detection)
dutchbay_bootstrap_rules.py	bootstrap	○	Tiny CLI helper to sanity-check the Go-with-the-Flow ruleset CSV: validates required columns and prints a rule-count/version summary.	RulesetSummary (dataclass), ruleset-validation main
run_full_pipeline_v14.py	entrypoint	●	Canonical Hydra CLI for the complete wind-to-finance pipeline: ingests frozen wind/solar exports, patches the scenario, and runs the lender-grade v14 finance engine with all report emitters.	main (hydra entrypoint), _load_wind_export, _load_solar_export, _run_wind_producer
run_scenario_analytics_v14.py	cli	●	Thin Hydra CLI for the lighter batch scenario-COMPARISON path (ScenarioAnalytics._run_single), ranking a directory of scenarios with informational-WACC KPIs; explicitly NOT the canonical lender engine.	main (hydra entrypoint)

legacy (2 — ●0 ●0 ○2)

Module	Kind	Cov	Purpose	Key API
__init__.py	legacy	○	Quarantine package marker for built-but-unwired modules retained for reference and excluded from coverage/SAST gates.	
stress_tests_v14.py	legacy	○	Quarantined deterministic stress-testing engine (rate/market/inflation shocks, combined severe) producing NPV/IRR impacts and stress-loss scalars mislabeled as VaR/CVaR; explicitly built-but-unwired.	StressTestEngine, StressResult, StressScenario

legacy_scripts (1 — ●0 ●0 ○1)

Module	Kind	Cov	Purpose	Key API
make_clean_zip.py	cli	○	Repository archiving utility that walks the tree and produces a filtered .zip plus a JSON manifest and human-readable concatenated snapshot of source files.	main, make_zip, create_manifest, create_concatenated_snapshot, parse_args

examples (1 — ●0 ●0 ○1)

Module	Kind	Cov	Purpose	Key API
monte_carlo_lender_pack_example.py	cli	○	Standalone example/demo CLI that loads a scenario YAML, runs the analytics.mc engine, builds CASPER lender risk blocks, and exports a Monte Carlo lender-pack Excel.	main, load_config, run_monte_carlo_simulation, generate_lender_pack

analysis_tools (1 — ●0 ●0 ○1)

Module	Kind	Cov	Purpose	Key API
__init__.py	sandbox	○	Package marker for research/analysis helper scripts (e.g. FX correlation analysis); explicitly not for production pipelines.	

`config` (1 — ●0 ●0 ○1)

Module	Kind	Cov	Purpose	Key API
`__init__.py`	config	○	Empty package marker making the top-level config/ directory an importable Python package.	